OS PROJECT REPORT

(Project Term 2025)

(SCHEDSIM)

(An Intelligent CPU Scheduling Simulator App)

Submitted by

Biswajeet Kumar, Yash Raj, Pawanjit Kumar

12526419,12503587, 12503425

K24LL

B.Tech Computer Science and Engineering

Submitted To:

(Sachin Kumar Chawla)

(Assistant Professor)

School of Computer Science

Lovely Professional University, Phagwara

TABLE OF CONTENTS

LIST OF FIGURES

1. INTRODUCTION

1.1. Problem Statement:-

Develop a simulator for CPU scheduling algorithms (FCFS, SJF, Round Robin, Priority Scheduling) with real-time visualizations. The simulator should allow users to input processes with arrival times, burst times, and priorities and visualize Gantt charts and performance metrics like average waiting time and turnaround time.

1.2. What's new in the system to be developed/Scope of work

The proposed system introduces an **innovative and educational platform** dedicated to showcasing the **CPU Scheduling Algorithms** in an interactive, visual, and user-friendly way. Unlike traditional textbook explanations or console-based simulations, this app provides a **complete learning and visualization experience** for understanding how different CPU scheduling algorithms work in real time.

The system covers **multiple scheduling techniques** including:

- **First Come First Serve (FCFS)**
- **Shortest Job First (SJF)**
- **Round Robin (RR)**
- **Priority Scheduling (PS)**
- **Shortest Remanning Time First(SRTF)**
- **Multilevel Queue**

Each algorithm is presented with **step-by-step process input**, automated computation of **waiting time (WT)** and **turnaround time (TAT)**, and a visually rich **Gantt chart** to illustrate process execution order and timing.

A key innovation of this system is its **modern Flutter-based interface**, designed for mobile devices, ensuring smooth interaction, real-time feedback, and easy navigation across multiple pages:

1. **Algorithm Selection Page** – Select from available scheduling algorithms.
2. **Info Page** – View the description and enter the number of processes.
3. **Process Input Form** – Enter process details such as PID, Arrival Time, Burst Time, Priority, or Time Quantum.

4. **Results Page** – Displays computed results, per-process metrics, and a **vertical Gantt Chart** for better visualization on mobile screens.

Unlike existing simulators that provide only textual outputs, this app offers a **dynamic vertical Gantt chart** showing real-time process execution flow with context switch markers, distinct process colors, and animated transitions for improved understanding.

The **scope of work** involves developing:

- A complete Flutter-based front-end with intuitive UI components.
- Backend logic for all major CPU scheduling algorithms.
- Automatic calculation of process metrics (WT, TAT, averages).
- Vertical Gantt chart visualization for mobile-friendly display.
- Modular and extensible architecture allowing addition of future algorithms (like SRTF, Priority-Preemptive) and export features (PDF/PNG).

The future enhancement scope includes integrating **interactive learning modules**, **process animation timelines**, and **real-time comparison** between algorithms for deeper analysis.

This project aims to serve as an **educational and demonstrative tool** for students and professionals studying Operating Systems, making complex scheduling concepts **easy, visual, and interactive** — transforming abstract theory into an engaging practical experience.

1.3. PROJECT ANALYSIS:-

Understanding how different CPU scheduling algorithms work is one of the most important yet complex concepts in Operating System studies. However, the tools and resources available to students for learning these algorithms are often **scattered, text-heavy, and non-interactive**, making it difficult to visualize how processes are actually executed by the CPU in real time.

Most existing CPU scheduling simulators are **either web-based console tools or theoretical examples** that lack user engagement, visualization, and practical interactivity. They focus only on producing numerical results such as waiting time or turnaround time

but fail to provide **intuitive Gantt chart representations** or a guided understanding of how the scheduling decision evolves step by step.

This creates a **clear gap between theory and visualization** — while students may learn the formulas and definitions of algorithms like FCFS, SJF, Round Robin, and Priority Scheduling, they often struggle to visualize how these algorithms differ in real execution. Additionally, the lack of a **mobile-friendly, user-centric interface** limits accessibility and real-time experimentation.

Many available tools are also **platform-dependent** (requiring desktop or web access) and **lack flexibility** for learning on the go. They do not provide proper validation, context switching visualization, or detailed per-process analysis, which are crucial for academic learning and demonstrations.

Thus, there is a **need for a single, well-structured, and interactive platform** that bridges this gap by combining theoretical understanding with practical visualization.
The proposed **CPU Scheduling Simulator App** addresses this need by offering:

- A **comprehensive simulation environment** for all major CPU scheduling algorithms (FCFS, SJF, RR, PS).
- A **modern Flutter-based mobile interface**, ensuring accessibility and intuitive navigation.
- **Dynamic vertical Gantt charts** for better mobile display and visual clarity.
- **Automated computation** of waiting and turnaround times with clear tabular results.
- **Educational accuracy** with an engaging design suited for students, educators, and researchers.

By integrating these features, the system not only enhances conceptual learning but also provides a **practical, real-time visualization tool** for understanding CPU process scheduling.

It aims to make learning more **interactive, accurate, and convenient**, effectively bridging the gap between theoretical study and practical application in Operating Systems.

1.4. PROJECT DEFINATION:-

In today's digital learning environment, while a wide range of theoretical material on **CPU Scheduling Algorithms** is available across books, PDFs, and websites, it is often **scattered, text-based, and lacks interactivity**.

Students and learners face difficulty in understanding how these algorithms actually **work in execution**, as most of the available tools only explain formulas or provide non-visual outputs.

Existing simulators or online platforms are either **command-line tools** or **web-based programs** that focus on computation alone, without offering a **clear, visual representation of process flow**. As a result, users are unable to fully grasp how different algorithms — such as **First Come First Serve (FCFS)**, **Shortest Job First (SJF)**, **Round Robin (RR)**, and **Priority Scheduling (PS)** — differ in execution sequence, waiting times, and turnaround performance.

Currently, there is **no dedicated mobile-friendly platform** that provides a **structured, interactive, and educational experience** for understanding CPU scheduling. This lack of visualization leads to fragmented learning, limited conceptual clarity, and reduced student engagement while studying Operating System concepts.

The problem, therefore, lies in the **absence of an integrated, user-friendly simulation platform** that can both calculate and visually demonstrate how processes are executed under different scheduling techniques — allowing learners to analyze performance and compare results easily.

The proposed **CPU Scheduling Simulator App** aims to address this problem by providing a **comprehensive and visually interactive solution** that brings together all major CPU scheduling algorithms in one place.

Through a guided workflow — from algorithm selection to process input and final Gantt chart visualization — the app enables users to:

- Input process details easily through a clean interface.
- Generate results automatically, showing waiting time, turnaround time, and averages.

- Visualize process execution dynamically using a **vertical Gantt chart** optimized for mobile display.

This system will make learning CPU scheduling **simpler, more visual, and engaging**, promoting a deeper understanding of algorithmic concepts and improving the educational experience for students, teachers, and operating system enthusiasts alike.

1.5. Project Overview:

This project aims to develop an interactive and educational **mobile application** that demonstrates the working of major **CPU Scheduling Algorithms** used in Operating Systems.

The app provides a **complete, step-by-step simulation** of popular scheduling techniques such as **First Come First Serve (FCFS)**, **Shortest Job First (SJF)**, **Round Robin (RR)**, and **Priority Scheduling (PS)**.

Designed using **Flutter**, the application focuses on creating a **simple, visually engaging, and user-friendly platform** where users can input process details, simulate scheduling, and view the results instantly through graphical **Gantt Chart representations**.

Each algorithm page includes process entry forms, automatic calculation of **Waiting Time (WT)** and **Turnaround Time (TAT)**, and a dynamic chart that visually illustrates the execution order of processes.

The main goal of this project is to make learning and understanding CPU scheduling concepts **easier, interactive, and more practical** for students and professionals. By providing both **numerical and visual outputs** in a single app, this project bridges the gap between theoretical knowledge and real-time process visualization — making it an excellent educational tool for learners of Operating Systems.

2. SYSTEM DESIGN

2.1. User Navigation System:

a. **Algorithm Selection:**

Users can begin by selecting a scheduling algorithm such as **First Come First Serve (FCFS)**, **Shortest Job First (SJF)**, **Round Robin (RR)**, or **Priority Scheduling (PS)** from a clean and organized home screen.

b. **Information & Input Navigation:**

After selecting an algorithm, the user is guided to an **Info Page** that provides a short description of the selected algorithm along with a form to input the **total number of processes**.

c. **Process Data Entry:**

Users can then navigate to the **Process Form Page**, where they can input detailed process information like **Process ID, Arrival Time, Burst Time**, and (if applicable) **Priority or Time Quantum** in a structured and simple format.

d. **Results and Visualization:**

Upon submission, users are taken to the **Results Page**, which displays:

- A detailed **tabular view** of all processes with their **Waiting Time** and **Turnaround Time**.
- A visually appealing **Vertical Gantt Chart** that represents the CPU execution timeline in a clear and easy-to-understand format.

e. **Seamless Flow:**

The navigation between all screens — from **Algorithm Selection → Info → Process Form → Results** — is smooth, intuitive, and consistent, ensuring a user-friendly experience throughout the simulation process.

2.2. Algorithm Management Module:

a. **Algorithm Data Processing:**

The module is responsible for **managing, executing, and processing** data related to different CPU scheduling algorithms such as **FCFS**, **SJF**, **Round Robin**, and **Priority Scheduling**.
It ensures that each algorithm runs with the correct logic to calculate **Waiting Time (WT)**, **Turnaround Time (TAT)**, and **Average Metrics** efficiently.

b. **Structured Process Handling:**

The system maintains **structured data** for each process entered by the user — including **Process ID**, **Arrival Time**, **Burst Time**, **Priority**, and **Time Quantum** (where applicable).
This structured data is then utilized to generate accurate **execution sequences** and to visualize them in the form of a **Gantt Chart** for better understanding and comparison.

2.3. Dynamic User Interface:

a. **Smooth and Fast Screen Transitions:**

The app provides **seamless navigation** between different screens — from algorithm selection to process input and result visualization.
All transitions are optimized for **speed, responsiveness, and clarity**, ensuring that users experience a smooth and lag-free flow throughout the simulation process.

b. **Interactive and Informative Design Elements:**

Each major feature, such as **Algorithm Cards**, **Input Fields**, and **Result Sections**, is designed using **interactive and visually appealing layouts**.
The use of modern Flutter widgets, animations, and color schemes (like the signature blue theme) enhances user engagement and makes the learning process more intuitive and enjoyable.

2.4. Planning and Requirements Analysis:

a. **Define Project Goals:**

The main goal of this project is to design and develop a **mobile-based CPU Scheduling Simulator** that helps users understand and visualize the working of various scheduling algorithms such as **First Come First Serve (FCFS)**, **Shortest Job First (SJF)**, **Round Robin (RR)**, and **Priority Scheduling (PS)**.

The app aims to provide an **interactive, educational, and user-friendly platform** that demonstrates process scheduling through step-by-step input, automatic computation, and dynamic Gantt chart visualization.

b. **Collect and Organize Functional Requirements:**

During the analysis phase, all key requirements were identified and structured to ensure smooth implementation.
This includes:

- Input fields for **Process ID, Arrival Time, Burst Time**, and **Priority/Quantum**.
- Automatic computation of **Waiting Time (WT)** and **Turnaround Time (TAT)**.
- A visually dynamic **Vertical Gantt Chart** for process visualization.
- Validation for incorrect or excessive input (e.g., process limit ≤ 12).
- Organized navigation between screens (Algorithm → Info → Input → Result).

3. DEVELOPMENT:-

3.1. Front-End Development (Flutter and Dart):

a. Develop a responsive and interactive UI using Flutter.
b. Implement state and category selection screens with smooth navigation and animations.
c. Ensure mobile-first, cross-platform compatibility (Android/iOS).

3.2. Back-End Development:

i. Back-End Development (Logic Layer and Data Handling):

a. **Algorithm Logic Implementation:**

Instead of using an external server or database, all back-end functionality is handled **within the Flutter app** itself.
The system processes user inputs locally to execute CPU scheduling algorithms such as **FCFS**, **SJF**, **Round Robin**, and **Priority Scheduling**.
Each algorithm's logic is implemented in Dart to calculate **Waiting Time**, **Turnaround Time**, and **Average Metrics** based on the process data provided by the user.

b. **Data Management and Computation Handling:**

All user-entered process data (like **Process ID, Arrival Time, Burst Time, Priority,** etc.) is temporarily stored in the app's memory for processing.
The results (timeline, waiting times, averages) are computed dynamically without the need for an external database.
This ensures **faster performance** and **offline accessibility**, making the app ideal for mobile-based learning environments.

3.3. Tools and Technologies:

3.3.1. Programming Languages:

a. Dart (for mobile app development)

3.3.2. Frameworks/Libraries:

a. Flutter (for cross-platform UI development)
b. REST API (for communication between frontend and backend)

3.3.3. Tools:

a. Visual Studio Code (for coding and development)
b. Postman (for API testing)
c. Git and GitHub (for version control)
d. Android Studio Emulator / Physical Device (for app testing)

3.4 Testing and Quality Assurance:

a. **Unit Testing of Flutter Components:**

Each **Flutter widget** and UI component was tested individually to ensure smooth functionality and responsiveness across different devices.
The app's forms, buttons, navigation flows, and input validators were tested thoroughly to confirm that data entry and transitions work correctly under various conditions (valid, invalid, and edge-case inputs).

b. **Algorithm Logic Verification:**

Since the app performs all scheduling computations internally (without an API), **logical testing** was done for each algorithm —
including **FCFS**, **SJF**, **Round Robin**, and **Priority Scheduling** — to verify the accuracy of results such as **Waiting Time (WT)**, **Turnaround Time (TAT)**, and average metrics.

Multiple test cases were created and manually compared against expected outputs to ensure algorithmic correctness.

c. **System Integration and Performance Testing:**

The complete flow — from **Algorithm Selection → Input Page → Process Form → Results Page** — was tested to ensure consistent data transfer and smooth navigation.

The app's overall **speed, animation transitions, and chart rendering performance** were optimized for a lag-free experience even with multiple process entries.

d. **Bug Fixing and Optimization:**

Minor layout inconsistencies, data validation issues, and UI glitches identified during testing were resolved.

Focus was given to improving user experience by ensuring **stable performance, accurate computations, and clean visual presentation** across all devices.

3.5 Software Requirement Analysis:-

The Cultural Heritage App must meet a set of functional and non-functional requirements to ensure rich content delivery, smooth user experience, and scalability for future updates.

4. REQUIREMENTS

4.1 Functional Requirements

a. Algorithm Selection and Navigation:

- Users must be able to choose from multiple CPU scheduling algorithms including FCFS, SJF, Round Robin, and Priority Scheduling.
- Each algorithm should have a dedicated information page explaining its concept and purpose before simulation begins.

b. Process Input Handling:

- Users should be able to enter process details such as Process ID, Arrival Time, Burst Time, and Priority/Time Quantum (where applicable).
- The input form should validate data to prevent invalid or empty entries (e.g., negative values, empty fields, or excessive process counts).

c. Computation and Result Generation:

- The app must accurately calculate Waiting Time (WT), Turnaround Time (TAT), and their averages for each algorithm.
- Results should be displayed in a tabular format with all process metrics clearly visible.

d. Dynamic Gantt Chart Visualization:

- The app should generate a vertical Gantt Chart dynamically based on the scheduling logic.
- Each process should be represented by a unique color, and context switch markers should appear clearly in the timeline.

e. Error Handling and Validation:

- If the user enters incorrect or missing data, the app must show meaningful error messages.
- Process limits should be validated (e.g., a maximum of 12 processes for mobile layout optimization).

f. Future Expansion (Optional):

- Support for Preemptive Algorithms (like SRTF or Preemptive Priority) may be added in future updates.
- Possible data persistence using local storage or backend APIs for saving simulation history.

4.2 Non-Functional Requirements

a. Performance:

- The app should compute results instantly after process input without any noticeable delay.
- Gantt Chart rendering and screen transitions must remain smooth even for the maximum number of processes.

b. Scalability:

- The codebase should be modular and maintainable, allowing new algorithms or visualization improvements to be added easily in future versions.
- Future versions can integrate cloud-based APIs for user data and performance tracking.

c. Usability:

- The interface should be clean, intuitive, and easy to navigate for users of all levels (students, educators, researchers).
- Texts, colors, and layouts must maintain clarity for educational readability.

d. Reliability:

- The app should function correctly under all valid input conditions and handle errors gracefully without crashing.
- All calculations (WT, TAT) must consistently match theoretical results across multiple test cases.

e. Compatibility:

- The application must work efficiently on both Android and iOS platforms.
- All UI components should be responsive and adaptive to different screen sizes and orientations.

f. Security (Future Implementation):

- If a backend is added in future versions, communication between the app and server should be secured using HTTPS and authenticated endpoints.
- Local storage (if implemented) must protect user data and avoid unauthorized access.

4.3 GENERAL DESCRIPTION:-

i. Deployment and Documentation:

a. **App Deployment:**

After final testing and quality checks, the **CPU Scheduling Simulator App** will be deployed as a **mobile application** for Android devices using the **Google Play Store** (or distributed as an APK for educational use).
The app is developed using **Flutter**, making it **cross-platform compatible** — hence it can also be packaged and deployed on the **Apple App Store (iOS)** in future updates.

b. **Offline Architecture with Future Cloud Integration:**

In the current version, the app runs entirely **offline**, performing all scheduling calculations and data handling locally on the device.

However, the architecture has been designed to allow **future integration of cloud-based features**, such as storing user activity, simulation history, or analytics through **APIs hosted on Vercel, AWS, or Firebase** if required in upcoming versions.

c. Documentation and User Guide:

Comprehensive **documentation** has been prepared to assist both users and developers.

- **User Guide:** Explains how to navigate through the app, enter process details, choose algorithms, and interpret results.
- **Developer Documentation:** Provides an overview of the app's structure, algorithm logic, and code modules for future enhancements.
- **Testing Records:** Include screenshots, sample input/output, and test case results to verify the accuracy of each algorithm's execution.

5. DESIGN DOCUMENTATION

5.1. Revision Tracking On Github

Use of GitHub in the Project

GitHub was used to maintain the **complete version history** of the *CPU Scheduler Simulator App*.

All source files — including the **Flutter front-end (UI)**, **Node.js REST API backend**, and **MySQL database schema** — were uploaded to a private GitHub repository.

Each update or improvement in the project was committed with descriptive messages to ensure proper tracking of changes throughout development.

5.1.1. Version Control

GitHub provided efficient version control for the entire project. Every revision of the code — from Flutter UI updates to backend API improvements — was properly stored in the repository.

In case of any error or issue, older stable versions could easily be restored without affecting the ongoing development.

5.1.2. Team Collaboration

If multiple contributors were involved, GitHub enabled smooth collaboration.

Developers could work on different parts of the app (for example, UI, backend routes, or database integration) without overwriting each other's code. Git automatically handled **merging** and **conflict resolution**, ensuring smooth teamwork.

5.1.3. Commit History

Every change made during the development cycle was documented through **commit messages**, helping track progress step-by-step.

Examples of actual commit messages include:

- "Added FCFS and Round Robin scheduling logic"
- "Updated API endpoint for process result calculation"
- "Improved Gantt chart UI for better visualization"
- "Integrated PDF export functionality in results page"

This clear commit history showed how the project evolved over time.

5.1.4. Branching and Testing

Different branches (like development, testing, and main) were used to manage the workflow.

New features were first implemented and tested in separate branches. Once verified, they were merged into the main branch to keep the main application stable and error-free.

5.1.5. Cloud Backup

All project files were securely stored on GitHub's cloud servers. Even if the local system was formatted or lost, the project could easily be cloned again from GitHub at any time.

This acted as a **safe backup** for the complete source code.

5.1.6. Continuous Improvement

The GitHub repository worked as a timeline that reflected the continuous improvement of the project — from the **initial prototype** to the **final polished version** with full functionality.

This ensured proper documentation of the project's entire development journey.

Repository Name: [cpu-scheduler-simulator],

GitHub Link: [https://github.com/Biswajeet111/cpu-scheduler-simulator.git]

5.2. DESIGN:-

The design phase focused on creating a clean, modern, and user-friendly interface for the CPU Scheduler Simulator App.

The entire UI layout was designed using Canva, which helped in visually planning the structure, color themes, and overall appearance of the application before actual implementation in Flutter.

The design process included the following stages:

5.2.1 UI Planning

Each screen of the application — such as the Algorithm Selection Page, Process Input Form, and Results Page — was first sketched in Canva to define the layout, spacing, and visual hierarchy.

This ensured that every element was positioned clearly and followed a consistent design pattern.

5.2.2 Color Scheme and Typography

A consistent blue and white color palette was chosen to give a clean and professional look to the interface.

Typography was kept minimal and readable to maintain clarity while displaying process tables and Gantt charts.

5.2.3. Component Design

Custom cards, buttons, and input fields were designed to look attractive yet simple. Rounded corners and shadow effects were applied to enhance visual appeal while maintaining simplicity and responsiveness.

5.2.4. Responsive Layout Planning

The Canva designs were created keeping in mind mobile-friendly proportions. During implementation, these layouts were translated into Flutter widgets using containers, padding, and scrollable views to ensure smooth rendering across different screen sizes.

5.2.5. Visual Consistency

All pages were designed in Canva following a uniform theme so that the user experience remained consistent throughout — from algorithm selection to result visualization (Gantt chart).

5.3. Er Diagram Of Table Design:-

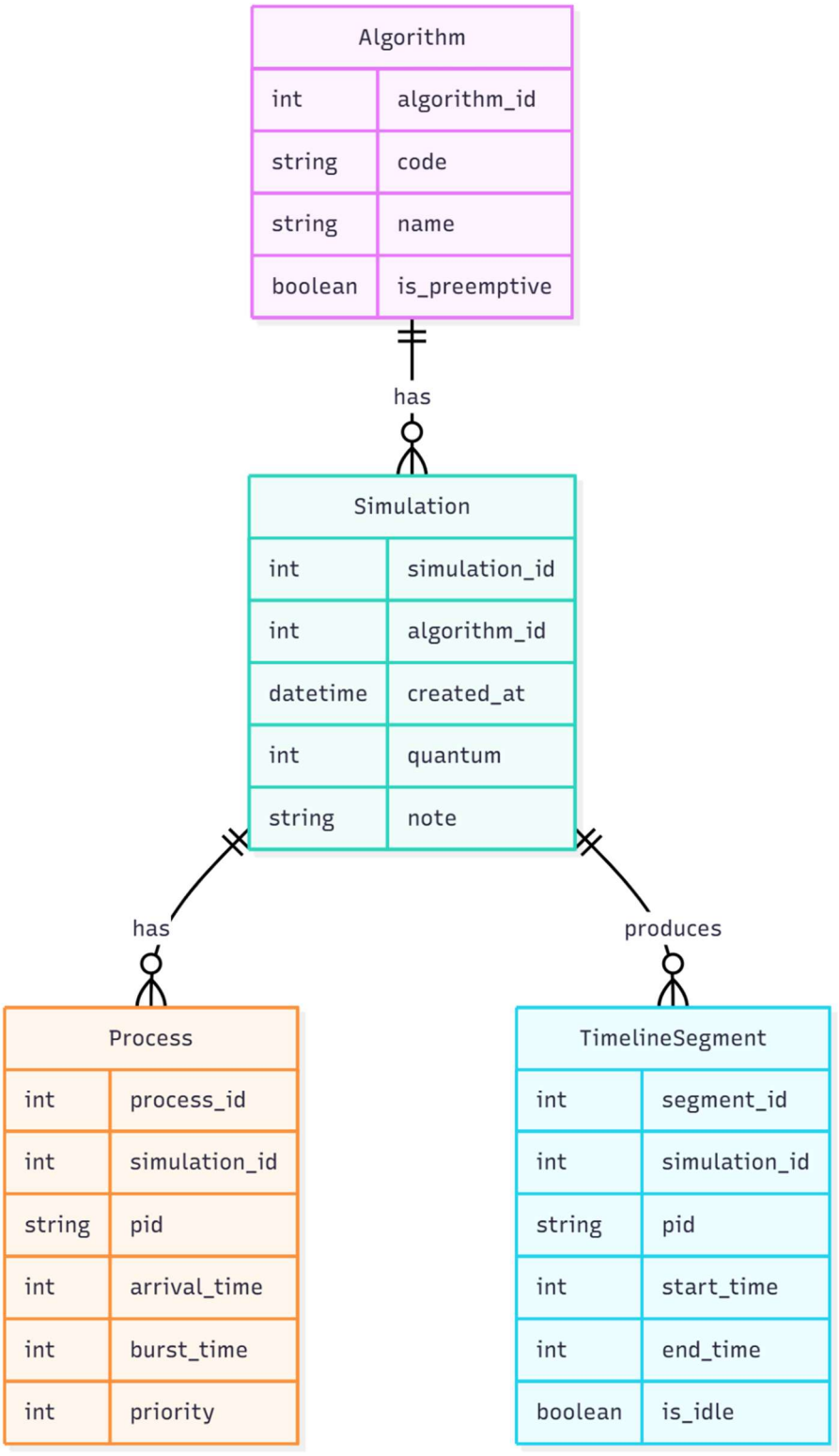

Fig. No. 1

5.2.2. DFD For Present System/Flowcharts In Both S/W And H/W Project

20.

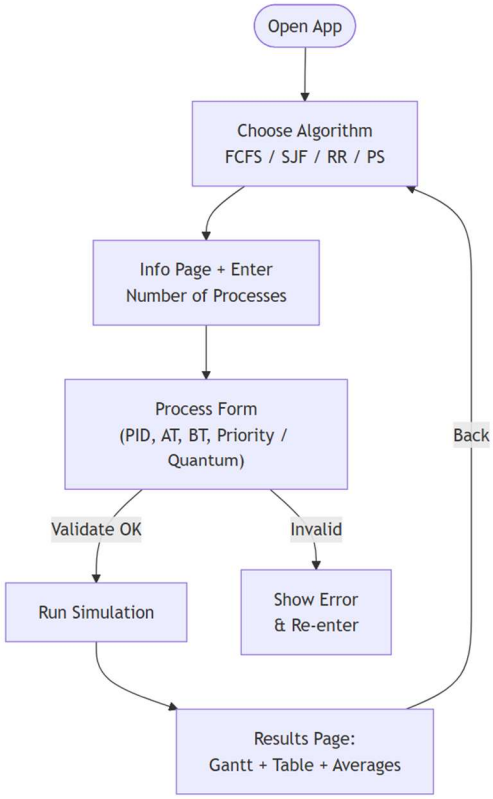

Fig. No. 2

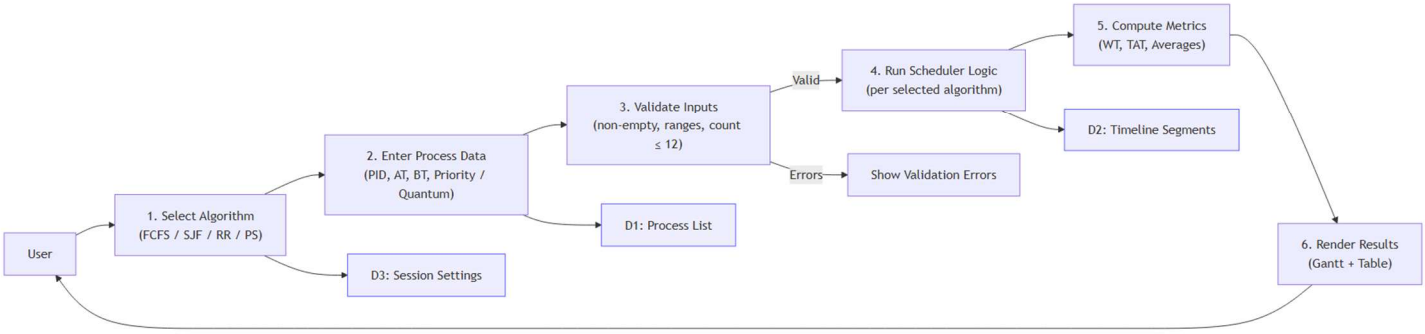

Fig. No. 3

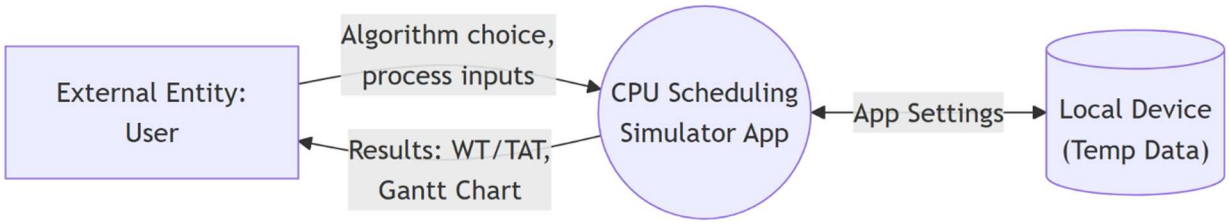

Fig. No. 4

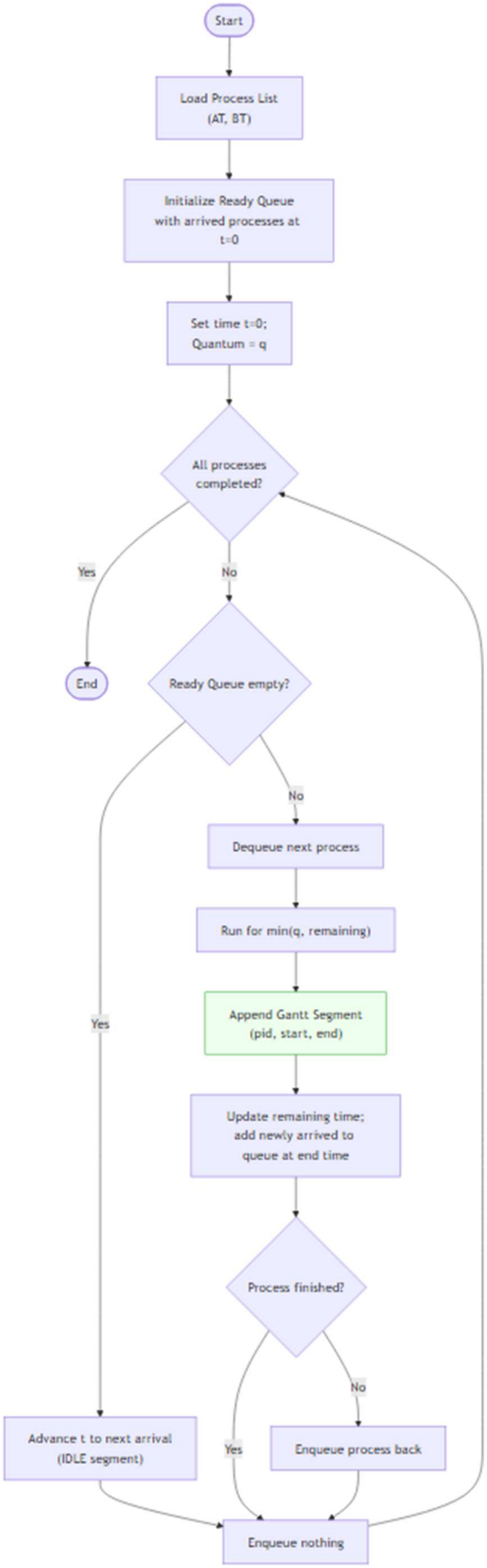
Start
Load Process List (AT, BT)
Initialize Ready Queue with arrived processes at t=0
Set time t=0; Quantum = q
All processes completed?
Yes
No
End
Ready Queue empty?
No
Dequeue next process
Run for min(q, remaining)
Append Gantt Segment (pid, start, end)
Update remaining time; add newly arrived to queue at end time
Process finished?
No
Yes
Enqueue process back
Yes
Advance t to next arrival (IDLE segment)
Enqueue nothing

Fig. No. 5

6. IMPLEMENTATION:-

6.1. Code implementation :-

Module 1. main.dart

```
import 'package:flutter/material.dart';
import 'starting_page.dart';
void main() {
 runApp(const CpuSchedulerApp());
}
class CpuSchedulerApp extends StatelessWidget {
 const CpuSchedulerApp({super.key});
 @override
 Widget build(BuildContext context) {
  return MaterialApp(
   debugShowCheckedModeBanner: false,
   title: 'CPU Scheduler Simulator',
   theme: ThemeData(
    primarySwatch: Colors.blue,
   ),
   home: starting_page(),
  );
 }
}
```

Module 2. starting_page.dart

```
// ignore_for_file: camel_case_types

import 'package:flutter/material.dart';
import 'ChooseAlgorithmPage.dart';

void main() => runApp(const starting_page());

class starting_page extends StatelessWidget {
 const starting_page({super.key});

 @override
 Widget build(BuildContext context) {
  const primaryBlue = Color(0xFF0A63C9);
  return MaterialApp(
   debugShowCheckedModeBanner: false,
   title: 'SCHEDSIM',
```

```
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(seedColor: primaryBlue),
        textTheme: const TextTheme(
          displayLarge: TextStyle(
            fontSize: 44,
            fontStyle: FontStyle.italic,
            fontWeight: FontWeight.w400,
            color: Colors.white,
          ),
          headlineMedium: TextStyle(
            fontSize: 36,
            fontWeight: FontWeight.bold,
            letterSpacing: 1.5,
            color: Color(0xFFD6E4FF),
          ),
          bodyLarge: TextStyle(
            fontSize: 18,
            height: 1.6,
            color: Colors.white,
          ),
        ),
      ),
      home: const SchedSimWelcomePage(),
    );
  }
}

class SchedSimWelcomePage extends StatelessWidget {
  const SchedSimWelcomePage({super.key});

  @override
  Widget build(BuildContext context) {
    const bg = Color(0xFF0A63C9);
    final size = MediaQuery.of(context).size;

    return Scaffold(
      backgroundColor: bg,
      body: SafeArea(
        child: Center(
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 28),
```

```
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.center,
              children: [
                SizedBox(height: size.height * 0.08),

                // Welcome text
                Align(
                  alignment: Alignment.center,
                  child: Text('Welcome to',
                      style: Theme.of(context).textTheme.displayLarge),
                ),
                const SizedBox(height: 25),

                //  🔹 Logo image
                Image.asset(
                  'assets/logo.png',
                  height: 300,
                  width: 300,
                  fit: BoxFit.contain,
                ),

                const SizedBox(height: 18),

                // App name
                // Text(
                //   'SCHEDSIM',
                //   style: Theme.of(context).textTheme.headlineMedium,
                // ),

                const SizedBox(height: 18),

                // Description
              Flexible(
  child: Text(
    'Explore and understand classic CPU scheduling algorithms like '
    'FCFS, SJF, Round Robin, and Priority Scheduling. Visualize
process '
    'execution, Gantt charts, waiting time, turnaround time — all in one
app!',
    textAlign: TextAlign.center,
    style: Theme.of(context).textTheme.bodyLarge!.copyWith(
          fontSize: 15,
```

```
          height: 1.4,
        ),
      ),
    ),
  ),

              const Spacer(),

              // Get Started Button
              SizedBox(
                width: double.infinity,
                child: ElevatedButton(
                  onPressed: () {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) => const ChooseAlgorithmPage(),
    ),
  );
},

                    style: ElevatedButton.styleFrom(
                      backgroundColor: Colors.white,
                      foregroundColor: bg,
                      padding: const EdgeInsets.symmetric(vertical: 18),
                      shape: RoundedRectangleBorder(
                        borderRadius: BorderRadius.circular(14),
                      ),
                      elevation: 0,
                    ),
                    child: const Row(
                      mainAxisAlignment: MainAxisAlignment.center,
                      children: [
                        Text(
                          'GET STARTED',
                          style: TextStyle(
                            fontSize: 15,
                            fontWeight: FontWeight.w700,
                            letterSpacing: 1.2,
                          ),
                        ),
                        SizedBox(width: 10),
                        Icon(Icons.arrow_forward_ios, size: 18),
```

```
                        ],
                      ),
                    ),
                  ),

                  const SizedBox(height: 20),
                ],
              ),
            ),
          ),
        ),
      );
    }
  }
```

Module 3. ChooseAlgorithmPage.dart

```
// ignore_for_file: file_names, unnecessary_underscores

import 'package:flutter/material.dart';
import 'info_page.dart';

class ChooseAlgorithmPage extends StatefulWidget {
  const ChooseAlgorithmPage({super.key});

  @override
  State<ChooseAlgorithmPage> createState() =>
_ChooseAlgorithmPageState();
}

class _ChooseAlgorithmPageState extends State<ChooseAlgorithmPage> {
  static const Color primaryBlue = Color(0xFF0A63C9);
  static const Color bgLight = Color(0xFFF3F6FB);

  // Full + short names
  final List<Map<String, String>> _algos = [
    {'short': 'FCFS', 'full': 'First Come First Serve'},
    {'short': 'SJF',  'full': 'Shortest Job First'},
    {'short': 'RR',   'full': 'Round Robin'},
    {'short': 'PS',   'full': 'Priority Scheduling'},
    {'short': 'SRTF', 'full': 'Shortest Remaining Time First'},
    {'short': 'MLQ',  'full': 'Multi-Level Queue Scheduling'},
  ];
```

```
int? _selectedIndex;

@override
Widget build(BuildContext context) {
 return Scaffold(
  backgroundColor: bgLight,
  appBar: AppBar(
   backgroundColor: primaryBlue,
   foregroundColor: Colors.white,
   title: const Text(
    'Select Algorithm',
    style: TextStyle(fontWeight: FontWeight.w700),
   ),
   centerTitle: true,
  ),
  body: ListView.separated(
   padding: const EdgeInsets.fromLTRB(20, 20, 20, 24),
   itemCount: _algos.length,
   separatorBuilder: (_, __) => const SizedBox(height: 16),
   itemBuilder: (context, i) {
    final algo = _algos[i];
    final sel = _selectedIndex == i;
    final label = '${algo['full']!} (${algo['short']!})';

    return InkWell(
     borderRadius: BorderRadius.circular(14),
     onTap: () {
      setState(() => _selectedIndex = i);
      Navigator.push(
       context,
       MaterialPageRoute(
        builder: (_) => InfoPage(
         algoName: algo['short']!,
         fullName: algo['full']!,
        ),
       ),
      );
     },
     child: Container(
      padding: const EdgeInsets.symmetric(horizontal: 18, vertical: 16),
      decoration: BoxDecoration(
       borderRadius: BorderRadius.circular(14),
```

```
              border: Border.all(color: primaryBlue, width: 2),
            ),
            child: Row(
              children: [
                Expanded(
                  child: Text(
                    label,
                    style: const TextStyle(
                      fontSize: 18,
                      fontWeight: FontWeight.w600,
                    ).copyWith(color: primaryBlue),
                  ),
                ),
                _RadioCircle(selected: sel),
              ],
            ),
          ),
        );
      },
    ),
  );
 }
}

class _RadioCircle extends StatelessWidget {
 const _RadioCircle({required this.selected});
 final bool selected;

 @override
 Widget build(BuildContext context) {
  const primaryBlue = Color(0xFF0A63C9);
  return Container(
   width: 28,
   height: 28,
   decoration: BoxDecoration(
    shape: BoxShape.circle,
    border: Border.all(color: primaryBlue, width: 2),
   ),
   alignment: Alignment.center,
   child: AnimatedContainer(
    duration: const Duration(milliseconds: 160),
    width: selected ? 14 : 0,
```

```
            height: selected ? 14 : 0,
            decoration: const BoxDecoration(
              color: primaryBlue,
              shape: BoxShape.circle,
            ),
          ),
        );
      }
    }
```

Module 4. info_page.dart

```
import 'package:flutter/material.dart';
import 'process_form_page.dart';

class InfoPage extends StatefulWidget {
  const InfoPage({
    super.key,
    required this.algoName, // short: FCFS/SJF/RR/PS
    required this.fullName, // full: First Come First Serve, ...
  });

  final String algoName;
  final String fullName;

  @override
  State<InfoPage> createState() => _InfoPageState();
}

class _InfoPageState extends State<InfoPage> {
  static const Color primaryBlue = Color(0xFF0A63C9);
  static const Color bgLight = Color(0xFFF3F6FB);

  final _formKey = GlobalKey<FormState>();
  final _controller = TextEditingController();

  /// Optional short descriptions keyed by SHORT name
  static const Map<String, String> _descriptions = {
    'FCFS': 'Processes execute in arrival order. Simple & non-
preemptive.',
    'SJF':  'Executes the job with the smallest CPU burst next.',
    'RR':   'Each process gets a fixed time quantum in round-robin
fashion.',
```

```
    'PS':   'Higher priority process runs first (preemptive/non-
preemptive).',
  };

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final short = widget.algoName.toUpperCase().trim();
    final full  = widget.fullName.trim();
    final desc  = _descriptions[short];

    return Scaffold(
      backgroundColor: bgLight,
      appBar: AppBar(
        backgroundColor: primaryBlue,
        foregroundColor: Colors.white,
        title: Text(
          '$full ($short)',
          style: const TextStyle(fontWeight: FontWeight.w700),
        ),
        centerTitle: true,
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.fromLTRB(24, 24, 24, 24),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                if (desc != null) ...[
                  Text(
                    desc,
                    style: const TextStyle(
                      color: primaryBlue,
                      fontSize: 16,
                      fontWeight: FontWeight.w500,
```

```
      height: 1.4,
    ),
  ),
  const SizedBox(height: 20),
],
Text(
  'You selected: $full ($short)',
  style: const TextStyle(
    color: primaryBlue,
    fontSize: 20,
    fontWeight: FontWeight.w600,
    height: 1.3,
  ),
),
const SizedBox(height: 16),

const Text(
  'Please enter the total number\nof processes:',
  style: TextStyle(
    color: primaryBlue,
    fontSize: 22,
    fontWeight: FontWeight.w600,
    height: 1.3,
  ),
),
const SizedBox(height: 24),

// Input field
Container(
  decoration: BoxDecoration(
    color: primaryBlue,
    borderRadius: BorderRadius.circular(12),
  ),
  padding: const EdgeInsets.symmetric(horizontal: 16),
  child: TextFormField(
    controller: _controller,
    keyboardType: TextInputType.number,
    style: const TextStyle(color: Colors.white, fontSize: 18),
    cursorColor: Colors.white,
    decoration: const InputDecoration(
      hintText: 'e.g. 4',
      hintStyle: TextStyle(color: Colors.white70),
```

```
      border: InputBorder.none,
    ),
    validator: (v) {
      if (v == null || v.trim().isEmpty) return 'Required';
      final n = int.tryParse(v.trim());
      if (n == null || n <= 0) return 'Enter a valid number';
      if (n > 12) return 'Keep ≤ 12 for mobile layout';
      return null;
    },
  ),
),

const SizedBox(height: 24),

SizedBox(
  width: double.infinity,
  child: ElevatedButton(
    style: ElevatedButton.styleFrom(
      backgroundColor: primaryBlue,
      foregroundColor: Colors.white,
      padding: const EdgeInsets.symmetric(vertical: 16),
    ),
    onPressed: () {
      if (!_formKey.currentState!.validate()) return;
      final count = int.parse(_controller.text.trim());
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (_) => ProcessFormPage(
            // Keep passing SHORT name forward for logic keys
            algoName: short,
            processCount: count,
          ),
        ),
      );
    },
    child: const Text(
      'SUBMIT',
      style: TextStyle(fontWeight: FontWeight.w700),
    ),
  ),
),
```

```
                    ],
                  ),
                ),
              ),
            ),
          );
        }
      }
```

Module 5. process_form_page.dart

```
// ignore_for_file: unused_local_variable

import 'package:flutter/material.dart';
import 'results_page.dart'; // Results UI (already added)

class ProcessFormPage extends StatefulWidget {
  const ProcessFormPage({
    super.key,
    required this.algoName,
    required this.processCount,
  });

  final String algoName; // 'FCFS', 'SJF', 'RR', 'PS'
  final int processCount;

  @override
  State<ProcessFormPage> createState() => _ProcessFormPageState();
}

class _ProcessFormPageState extends State<ProcessFormPage> {
  static const Color primaryBlue = Color(0xFF0A63C9);
  static const Color bgLight = Color(0xFFF3F6FB);

  late final List<TextEditingController> pidCtrls;
  late final List<TextEditingController> atCtrls;
  late final List<TextEditingController> btCtrls;
  late final List<TextEditingController>? prCtrls; // only for PS
  final TextEditingController _quantumCtrl = TextEditingController(); // only
for RR

  bool get isPS => widget.algoName.toUpperCase() == 'PS';
  bool get isRR => widget.algoName.toUpperCase() == 'RR';
```

```
  @override
  void initState() {
    super.initState();
    pidCtrls = List.generate(widget.processCount, (i) =>
TextEditingController(text: 'P${i + 1}'));
    atCtrls  = List.generate(widget.processCount, (i) =>
TextEditingController());
    btCtrls  = List.generate(widget.processCount, (i) =>
TextEditingController());
    prCtrls  = isPS ? List.generate(widget.processCount, (i) =>
TextEditingController()) : null;
  }

  @override
  void dispose() {
    for (final c in [...pidCtrls, ...atCtrls, ...btCtrls, if (prCtrls != null)
...prCtrls!]) {
      c.dispose();
    }
    _quantumCtrl.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final algo = widget.algoName.toUpperCase();

    return Scaffold(
      backgroundColor: bgLight,
      appBar: AppBar(
        backgroundColor: primaryBlue,
        foregroundColor: Colors.white,
        title: Text(algo, style: const TextStyle(fontWeight: FontWeight.w700)),
        centerTitle: true,
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.fromLTRB(20, 20, 20, 24),
          child: Column(crossAxisAlignment: CrossAxisAlignment.start,
children: [
            const Text(
              'Enter the process details\nbelow to start the simulation.',
```

```
            style: TextStyle(color: primaryBlue, fontSize: 22, fontWeight:
FontWeight.w600, height: 1.35),
          ),
          const SizedBox(height: 20),

          // Algo chip
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
            decoration: BoxDecoration(color: primaryBlue, borderRadius:
BorderRadius.circular(12)),
            child: Row(children: [
              Expanded(
                child: Text(algo,
                    style: const TextStyle(color: Colors.white, fontSize: 16,
fontWeight: FontWeight.w600)),
              ),
              const Icon(Icons.check_circle_rounded, color: Colors.white, size:
22),
            ]),
          ),

          if (isRR) ...[
            const SizedBox(height: 16),
            const Text('Time Quantum', style: TextStyle(color: primaryBlue,
fontSize: 16, fontWeight: FontWeight.w700)),
            const SizedBox(height: 8),
            _blueInput(_quantumCtrl, hint: 'e.g. 2'),
          ],

          const SizedBox(height: 18),

          // Headers
          Row(
            children: [
              _header('PID'),
              _header('Arrival Time'),
              _header('Burst Time'),
              if (isPS) _header('Priority'),
            ],
          ),
          const SizedBox(height: 8),
```

```
          // Columns
          Row(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Expanded(child: _blueColumn(pidCtrls, hint: 'P1', isNumber:
false)),
              const SizedBox(width: 12),
              Expanded(child: _blueColumn(atCtrls, hint: '0')),
              const SizedBox(width: 12),
              Expanded(child: _blueColumn(btCtrls, hint: '5')),
              if (isPS) ...[
                const SizedBox(width: 12),
                Expanded(child: _blueColumn(prCtrls!, hint: '1')), // lower number
= higher priority
              ]
            ],
          ),

          const SizedBox(height: 26),
          SizedBox(
            width: double.infinity,
            child: ElevatedButton(
              style: ElevatedButton.styleFrom(
                backgroundColor: primaryBlue,
                foregroundColor: Colors.white,
                padding: const EdgeInsets.symmetric(vertical: 18),
                shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(12)),
              ),
              onPressed: _onSubmit,
              child: const Text('SUBMIT', style: TextStyle(fontWeight:
FontWeight.w700)),
            ),
          ),
        ]),
      ),
    ),
  );
 }

 Widget _header(String text) => Expanded(
      child: Text(text,
```

```
          style: const TextStyle(color: primaryBlue, fontSize: 16, fontWeight:
FontWeight.w700)),
    );

  Widget _blueInput(TextEditingController c, {String? hint, bool isNumber =
true}) {
    return Container(
      decoration: BoxDecoration(color: primaryBlue, borderRadius:
BorderRadius.circular(12)),
      padding: const EdgeInsets.symmetric(horizontal: 16),
      child: TextField(
        controller: c,
        keyboardType: isNumber ? TextInputType.number : TextInputType.text,
        style: const TextStyle(color: Colors.white, fontSize: 16),
        cursorColor: Colors.white,
        decoration: InputDecoration(
          hintText: hint,
          hintStyle: const TextStyle(color: Colors.white70),
          border: InputBorder.none,
        ),
      ),
    );
  }

  Widget _blueColumn(List<TextEditingController> ctrls, {String? hint, bool
isNumber = true}) {
    return Column(
      children: List.generate(ctrls.length, (i) {
        return Padding(
          padding: const EdgeInsets.symmetric(vertical: 6),
          child: Container(
            decoration: BoxDecoration(color: primaryBlue, borderRadius:
BorderRadius.circular(20)),
            padding: const EdgeInsets.symmetric(horizontal: 14),
            child: TextField(
              controller: ctrls[i],
              keyboardType: isNumber ? TextInputType.number :
TextInputType.text,
              style: const TextStyle(color: Colors.white, fontSize: 16),
              cursorColor: Colors.white,
              decoration: InputDecoration(
                hintText: hint,
```

```
              hintStyle: const TextStyle(color: Colors.white70),
              border: InputBorder.none,
            ),
          ),
        ),
      );
    }),
  );
 }

 // ==================== SUBMIT: parse -> compute -> navigate ====================

 void _onSubmit() {
  final algo = widget.algoName.toUpperCase();
  final n = widget.processCount;

  // Parse & validate
  final ids = <String>[];
  final at  = <int>[];
  final bt  = <int>[];
  final pr  = <int>[];

  for (int i = 0; i < n; i++) {
   final id = (pidCtrls[i].text.trim().isEmpty) ? 'P${i + 1}' : pidCtrls[i].text.trim();
   final a = int.tryParse(atCtrls[i].text.trim());
   final b = int.tryParse(btCtrls[i].text.trim());
   if (a == null || b == null) { _toast('Row ${i + 1}: Enter valid Arrival/Burst'); return; }
   if (b <= 0) { _toast('Row ${i + 1}: Burst must be > 0'); return; }
   ids.add(id); at.add(a); bt.add(b);
  }

  int? quantum;
  if (isRR) {
   quantum = int.tryParse(_quantumCtrl.text.trim());
   if (quantum == null || quantum <= 0) { _toast('Enter a valid Time Quantum (>0)'); return; }
  }

  if (isPS) {
```

```
  for (int i = 0; i < n; i++) {
    final p = int.tryParse(prCtrls![i].text.trim());
    if (p == null) { _toast('Row ${i + 1}: Enter valid Priority'); return; }
    pr.add(p);
  }
}

// Build table rows for Results
final rows = <ResultRow>[
  for (int i = 0; i < n; i++) ResultRow(pid: ids[i], arrival: at[i], burst: bt[i]),
];

// Run selected algorithm
_AlgoOutput out;
switch (algo) {
  case 'FCFS':
    out = _fcfs(ids, at, bt);
    break;
  case 'SJF':
    out = _sjfNonPreemptive(ids, at, bt);
    break;
  case 'PS':
    out = _priorityNonPreemptive(ids, at, bt, pr);
    break;
  case 'RR':
    out = _roundRobin(ids, at, bt, quantum!);
    break;
  default:
    _toast('Unknown algorithm: $algo'); return;
}

// Navigate to Results page (UI only)
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => ResultsPage(
      algoName: algo,
      rows: rows,
      avgWaiting: out.avgWaiting,
      avgTurnaround: out.avgTurnaround,
      timeline: out.timeline,
    ),
```

```
      ),
    );
  }

  void _toast(String msg) {
    ScaffoldMessenger.of(context).showSnackBar(SnackBar(content:
Text(msg)));
  }
}

// ----------------- Helper models (top-level) -----------------
class _Proc {
  final String id;
  final int arrival;
  final int burst;
  final int? priority;
  _Proc(this.id, this.arrival, this.burst, {this.priority});
}

class _AlgoOutput {
  final List<GanttSegment> timeline; // ordered segments
  final double avgWaiting;
  final double avgTurnaround;
  _AlgoOutput(this.timeline, this.avgWaiting, this.avgTurnaround);
}

// ----------------- Algorithms -----------------

// FCFS
_AlgoOutput _fcfs(List<String> id, List<int> at, List<int> bt) {
  final procs = <_Proc>[
    for (int i = 0; i < id.length; i++) _Proc(id[i], at[i], bt[i]),
  ]..sort((a, b) => a.arrival.compareTo(b.arrival));

  final timeline = <GanttSegment>[];
  final finish = <String, int>{};
  int time = 0;

  for (final p in procs) {
    if (time < p.arrival) {
      timeline.add(GanttSegment(pid: 'IDLE', start: time, end: p.arrival, color:
const Color(0xFFB0B8C1)));
```

```
      time = p.arrival;
    }
    timeline.add(GanttSegment(pid: p.id, start: time, end: time + p.burst));
    time += p.burst;
    finish[p.id] = time;
  }

  final (avgW, avgT) = _averages(procs, finish);
  return _AlgoOutput(timeline, avgW, avgT);
}

// SJF (Non-preemptive)
_AlgoOutput _sjfNonPreemptive(List<String> id, List<int> at, List<int> bt)
{
  final all = <_Proc>[
    for (int i = 0; i < id.length; i++) _Proc(id[i], at[i], bt[i]),
  ]..sort((a, b) => a.arrival.compareTo(b.arrival));

  final timeline = <GanttSegment>[];
  final finish = <String, int>{};
  int time = 0;
  final ready = <_Proc>[];
  int idx = 0;

  while (idx < all.length || ready.isNotEmpty) {
    while (idx < all.length && all[idx].arrival <= time) {
      ready.add(all[idx++]);
    }
    if (ready.isEmpty) {
      final next = all[idx];
      timeline.add(GanttSegment(pid: 'IDLE', start: time, end: next.arrival,
color: const Color(0xFFB0B8C1)));
      time = next.arrival;
      continue;
    }
    // shortest burst
    ready.sort((a, b) => a.burst.compareTo(b.burst));
    final p = ready.removeAt(0);
    timeline.add(GanttSegment(pid: p.id, start: time, end: time + p.burst));
    time += p.burst;
    finish[p.id] = time;
  }
```

```
  final (avgW, avgT) = _averages(all, finish);
  return _AlgoOutput(timeline, avgW, avgT);
}

// Priority (Non-preemptive; smaller number = higher priority)
_AlgoOutput _priorityNonPreemptive(List<String> id, List<int> at, List<int>
bt, List<int> pr) {
  final all = <_Proc>[
    for (int i = 0; i < id.length; i++) _Proc(id[i], at[i], bt[i], priority: pr[i]),
  ]..sort((a, b) => a.arrival.compareTo(b.arrival));

  final timeline = <GanttSegment>[];
  final finish = <String, int>{};
  int time = 0;
  final ready = <_Proc>[];
  int idx = 0;

  while (idx < all.length || ready.isNotEmpty) {
    while (idx < all.length && all[idx].arrival <= time) {
      ready.add(all[idx++]);
    }
    if (ready.isEmpty) {
      final next = all[idx];
      timeline.add(GanttSegment(pid: 'IDLE', start: time, end: next.arrival,
color: const Color(0xFFB0B8C1)));
      time = next.arrival;
      continue;
    }
    // highest priority first
    ready.sort((a, b) {
      final c = (a.priority ?? 0).compareTo(b.priority ?? 0);
      return c != 0 ? c : a.arrival.compareTo(b.arrival);
    });
    final p = ready.removeAt(0);
    timeline.add(GanttSegment(pid: p.id, start: time, end: time + p.burst));
    time += p.burst;
    finish[p.id] = time;
  }

  final (avgW, avgT) = _averages(all, finish);
  return _AlgoOutput(timeline, avgW, avgT);
```

```
}

// Round Robin (preemptive)
_AlgoOutput _roundRobin(List<String> id, List<int> at, List<int> bt, int q) {
  final n = id.length;
  final remaining = <String, int>{for (int i = 0; i < n; i++) id[i]: bt[i]};
  final arrival = <String, int>{for (int i = 0; i < n; i++) id[i]: at[i]};

  final order = List<int>.generate(n, (i) => i)..sort((a, b) =>
at[a].compareTo(at[b]));
  int time = 0;
  int nextIdx = 0;
  final queue = <String>[];
  final finish = <String, int>{};
  final timeline = <GanttSegment>[];

  void addArrivals() {
    while (nextIdx < n && at[order[nextIdx]] <= time) {
      queue.add(id[order[nextIdx]]);
      nextIdx++;
    }
  }

  if (nextIdx < n) {
    time = at[order[nextIdx]];
    addArrivals();
  }

  while (queue.isNotEmpty || nextIdx < n) {
    if (queue.isEmpty) {
      final jump = at[order[nextIdx]];
      if (time < jump) {
        timeline.add(GanttSegment(pid: 'IDLE', start: time, end: jump, color:
const Color(0xFFB0B8C1)));
        time = jump;
      }
      addArrivals();
      continue;
    }

    final pid = queue.removeAt(0);
    final rem = remaining[pid]!;
```

```
      final slice = rem > q ? q : rem;
      final start = time;
      final end = time + slice;
      timeline.add(GanttSegment(pid: pid, start: start, end: end));
      time = end;
      remaining[pid] = rem - slice;

      addArrivals();

      if (remaining[pid]! > 0) {
        queue.add(pid);
      } else {
        finish[pid] = time;
      }
    }

    final procs = <_Proc>[
      for (int i = 0; i < n; i++) _Proc(id[i], at[i], bt[i]),
    ];
    final (avgW, avgT) = _averages(procs, finish);
    return _AlgoOutput(timeline, avgW, avgT);
  }

  // Averages helper
  (double, double) _averages(List<_Proc> procs, Map<String, int> finish) {
    double totW = 0, totT = 0;
    for (final p in procs) {
      final c = finish[p.id] ?? 0;
      final tat = c - p.arrival;
      final wt = tat - p.burst;
      totT += tat;
      totW += wt;
    }
    final n = procs.length;
    return (totW / n, totT / n);
  }
```

Module 6. results_page.dart

```
// ignore_for_file: unused_element_parameter, unused_element,
deprecated_member_use, no_leading_underscores_for_local_identifiers,
use_build_context_synchronously, unused_import

import 'dart:io';
```

```
import 'package:flutter/material.dart';
import 'package:pdf/pdf.dart';
import 'package:pdf/widgets.dart' as pw;
import 'package:path_provider/path_provider.dart';
import 'package:permission_handler/permission_handler.dart';
import 'package:printing/printing.dart'; // optional, used for conversion helpers

/// ---------- DATA passed from ProcessFormPage ----------
class ResultRow {
  final String pid;
  final int arrival;
  final int burst;
  const ResultRow({required this.pid, required this.arrival, required this.burst});
}

class GanttSegment {
  final String pid;
  final int start; // inclusive
  final int end; // exclusive
  final Color? color;
  const GanttSegment({
    required this.pid,
    required this.start,
    required this.end,
    this.color,
  });
}

/// ---------- RESULTS PAGE (now Stateful to support export) ----------
class ResultsPage extends StatefulWidget {
  const ResultsPage({
    super.key,
    required this.algoName,
    required this.rows,
    required this.avgWaiting,
    required this.avgTurnaround,
    required this.timeline,
  });

  final String algoName;
  final List<ResultRow> rows;
  final double avgWaiting;
```

```
  final double avgTurnaround;
  final List<GanttSegment> timeline;

  @override
  State<ResultsPage> createState() => _ResultsPageState();
}

class _ResultsPageState extends State<ResultsPage> {
  static const Color primaryBlue = Color(0xFF0A63C9);
  static const Color bgLight = Color(0xFFF3F6FB);

  bool _saving = false;

  @override
  Widget build(BuildContext context) {
    final perProc = _computePerProcessMetrics(widget.rows, widget.timeline);
    final ganttData = widget.timeline
        .map((g) => {'pid': g.pid, 'start': g.start, 'end': g.end})
        .toList();

    return Scaffold(
      backgroundColor: bgLight,
      appBar: AppBar(
        backgroundColor: primaryBlue,
        foregroundColor: Colors.white,
        title: const Text('RESULTS', style: TextStyle(fontWeight: FontWeight.w700)),
        centerTitle: true,
        actions: [
          IconButton(
            icon: _saving
                ? const SizedBox(width: 20, height: 20, child:
CircularProgressIndicator(color: Colors.white, strokeWidth: 2))
                : const Icon(Icons.download),
            onPressed: _saving ? null : () => _onDownloadPressed(perProc,
widget.timeline),
            tooltip: 'Download PDF',
          )
        ],
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.fromLTRB(20, 20, 20, 28),
```

```
        child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
          const Text(
            'Review the Gantt chart and detailed process metrics below.',
            style: TextStyle(
                color: primaryBlue, fontSize: 22, fontWeight: FontWeight.w600, height:
1.35),
          ),
          const SizedBox(height: 16),

          // Algorithm chip
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
            decoration: BoxDecoration(color: primaryBlue, borderRadius:
BorderRadius.circular(12)),
            child: Row(children: [
              Expanded(
                child: Text(widget.algoName.toUpperCase(),
                    style: const TextStyle(color: Colors.white, fontSize: 16, fontWeight:
FontWeight.w600)),
              ),
              const Icon(Icons.check_circle_rounded, size: 22, color: Colors.white),
            ]),
          ),
          const SizedBox(height: 20),

          // Table header
          Row(children: const [
            _Header('PID'),
            _Header('Arrival Time'),
            _Header('Burst Time'),
            _Header('Waiting Time'),
            _Header('Turnaround Time'),
          ]),
          const SizedBox(height: 6),

          // Table
          _ResultGrid(rows: widget.rows, perProc: perProc),

          const SizedBox(height: 22),

          // Averages
          Center(
```

```
            child: Column(children: [
              Text('Average Waiting Time: ${widget.avgWaiting.toStringAsFixed(2)}',
                  style: const TextStyle(color: primaryBlue, fontSize: 18, fontWeight:
FontWeight.w700)),
              const SizedBox(height: 6),
              Text('Average Turnaround Time:
${widget.avgTurnaround.toStringAsFixed(2)}',
                  style: const TextStyle(color: primaryBlue, fontSize: 18, fontWeight:
FontWeight.w700)),
            ]),
          ),

          const SizedBox(height: 24),

          if (widget.timeline.isNotEmpty) ...[
            const Text("📊 Gantt Chart",
                style: TextStyle(color: primaryBlue, fontWeight: FontWeight.w700,
fontSize: 18)),
            const SizedBox(height: 10),
            _PrettyGanttChart(result: ganttData),
          ],
          const SizedBox(height: 20),

          // Big Download button (redundant to AppBar icon)
          SizedBox(
  width: double.infinity,
  child: ElevatedButton.icon(
    icon: Icon(Icons.download, color: const Color.fromARGB(255, 255, 255, 255)),
//  🔹 icon color blue
    label: Text(
      _saving ? 'Saving...' : 'Download PDF',
      style: const TextStyle(
        color: Color.fromARGB(255, 255, 255, 255),
        fontWeight: FontWeight.w600,
        fontSize: 16,
      ),
    ),
    style: ElevatedButton.styleFrom(
      backgroundColor: primaryBlue, //  🔹 background white
      padding: const EdgeInsets.symmetric(vertical: 14),
      shape: RoundedRectangleBorder(
```

```
        borderRadius: BorderRadius.circular(12),
        side: const BorderSide(color: primaryBlue, width: 1.2), // optional blue border
      ),
      elevation: 2, // soft shadow
    ),
    onPressed: _saving ? null : () => _onDownloadPressed(perProc, widget.timeline),
  ),
),

          ]),
        ),
      ),
    );
  }

  Future<void> _onDownloadPressed(List<_PerProc> perProc,
List<GanttSegment> timeline) async {
    setState(() => _saving = true);
    try {
      // Ask for storage permission (Android)
      final status = await Permission.storage.request();
      if (!status.isGranted) {
        // On Android 11+, WRITE_EXTERNAL_STORAGE may be restricted; still
we try saving to app dir
        ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content:
Text('Storage permission not granted. Will try app directory.')));
      }

      final pdfBytes = await _buildPdfBytes(widget.algoName, widget.rows, perProc,
widget.avgWaiting, widget.avgTurnaround, timeline);

      final savedPath = await _saveFileBytes(pdfBytes,
'schedsim_results_${DateTime.now().millisecondsSinceEpoch}.pdf');

      if (savedPath != null) {
        ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Saved
PDF to: $savedPath')));
      } else {
        ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content:
Text('Failed to save PDF.')));
      }
    } catch (e) {
```

```
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Error:
$e')));
    } finally {
      setState(() => _saving = false);
    }
  }

  /// Build the PDF file bytes using the `pdf` package.
  Future<List<int>> _buildPdfBytes(
    String algo,
    List<ResultRow> rows,
    List<_PerProc> perProc,
    double avgW,
    double avgT,
    List<GanttSegment> timeline,
  ) async {
    final doc = pw.Document();

    // Convert Flutter Color to PdfColor
    PdfColor _toPdfColor(Color c) => PdfColor.fromInt((c.value & 0xFFFFFFFF));

    // Simple table rows for PDF
    final tableHeaders = ['PID', 'Arrival', 'Burst', 'Waiting', 'Turnaround'];

    final tableData = [
      for (int i = 0; i < rows.length; i++)
        [
          rows[i].pid,
          rows[i].arrival.toString(),
          rows[i].burst.toString(),
          perProc.length > i ? perProc[i].wt.toString() : '',
          perProc.length > i ? perProc[i].tat.toString() : ''
        ]
    ];

    // Build Gantt drawing: normalize times into units for page width
    final int chartStart = timeline.isEmpty ? 0 : timeline.first.start;
    final int chartEnd = timeline.isEmpty ? 0 : timeline.last.end;
    final int chartTotal = (chartEnd - chartStart).clamp(1, 1 << 30);

    // Page
    doc.addPage(
```

```
    pw.MultiPage(
      pageFormat: PdfPageFormat.a4,
      build: (pw.Context ctx) {
        return [
          pw.Header(level: 0, child: pw.Text('SCHEDSIM Results — $algo', style:
pw.TextStyle(fontSize: 20))),
          pw.SizedBox(height: 6),
          pw.Text('Average Waiting Time: ${avgW.toStringAsFixed(2)}'),
          pw.Text('Average Turnaround Time: ${avgT.toStringAsFixed(2)}'),
          pw.SizedBox(height: 12),

          // Table
          pw.Table.fromTextArray(
            headers: tableHeaders,
            data: tableData,
            headerStyle: pw.TextStyle(fontWeight: pw.FontWeight.bold),
            cellAlignment: pw.Alignment.centerLeft,
            headerDecoration: pw.BoxDecoration(color: PdfColors.blue100),
            cellHeight: 20,
          ),

          pw.SizedBox(height: 18),

          // Gantt title
          pw.Text('Gantt Chart (timeline)', style: pw.TextStyle(fontSize: 14,
fontWeight: pw.FontWeight.bold)),
          pw.SizedBox(height: 8),

          // Draw a custom Gantt using pw.Container and pw.Positioned inside
pw.Stack-like approach
          pw.Container(
            height: 80,
            width: double.infinity,
            child: pw.Stack(
              children: [
                // vertical ticks
                for (int t = 0; t <= chartTotal; t++)
                  pw.Positioned(
                    left: (t / chartTotal) * (PdfPageFormat.a4.availableWidth - 40),
                    top: 0,
                    child: pw.Container(width: 1, height: 80, decoration:
pw.BoxDecoration(color: t % 5 == 0 ? PdfColors.grey600 : PdfColors.grey300)),
```

```
                ),

                // bars (single row stacked vertically)
                for (int i = 0; i < timeline.length; i++)
                  pw.Positioned(
                    left: ((timeline[i].start - chartStart) / chartTotal) *
(PdfPageFormat.a4.availableWidth - 40),
                    top: 12,
                    child: pw.Container(
                      width: ((timeline[i].end - timeline[i].start) / chartTotal) *
(PdfPageFormat.a4.availableWidth - 40),
                      height: 28,
                      decoration: pw.BoxDecoration(
                        borderRadius: pw.BorderRadius.all(pw.Radius.circular(8)),
                        gradient: pw.LinearGradient(
                          colors: [PdfColors.blue300, PdfColors.blue800],
                          begin: pw.Alignment.centerLeft,
                          end: pw.Alignment.centerRight,
                        ),
                      ),
                      child: pw.Center(child: pw.Text(timeline[i].pid, style:
pw.TextStyle(color: PdfColors.white, fontWeight: pw.FontWeight.bold))),
                    ),
                  ),
              ],
            ),
          ),
        ];
      },
    ),
  );

  return doc.save();
 }

 /// Save bytes to Downloads (if possible) or app external dir.
 Future<String?> _saveFileBytes(List<int> bytes, String filename) async {
  try {
    // Try Downloads directory (Android)
    if (Platform.isAndroid) {
      try {
        final extDir = await getExternalStorageDirectory();
```

```
        // extDir path is like /storage/emulated/0/Android/data/<package>/files
        // Try to save in a "Download" path if possible:
        final downloadsDir = Directory('/storage/emulated/0/Download');
        if (await downloadsDir.exists()) {
          final file = File('${downloadsDir.path}/$filename');
          await file.writeAsBytes(bytes);
          return file.path;
        } else if (extDir != null) {
          final file = File('${extDir.path}/$filename');
          await file.writeAsBytes(bytes);
          return file.path;
        }
      } catch (_) {
        // fallback below
      }
    }

    // For iOS or fallback: save in app documents directory
    final docDir = await getApplicationDocumentsDirectory();
    final file = File('${docDir.path}/$filename');
    await file.writeAsBytes(bytes);
    return file.path;
  } catch (e) {
    return null;
  }
}

/// Compute per-process metrics like WT/TAT using the end time from timeline.
List<_PerProc> _computePerProcessMetrics(List<ResultRow> rows,
List<GanttSegment> segs) {
  final finish = <String, int>{};
  for (final s in segs) {
    if (s.pid == 'IDLE') continue;
    final prev = finish[s.pid];
    if (prev == null || s.end > prev) finish[s.pid] = s.end;
  }

  final list = <_PerProc>[];
  for (final r in rows) {
    final c = finish[r.pid] ?? 0;
    final tat = c - r.arrival;
    final wt = tat - r.burst;
```

```
      list.add(_PerProc(pid: r.pid, wt: wt, tat: tat));
    }
    return list;
  }
}

/// ---------- Table Header ----------
class _Header extends StatelessWidget {
  const _Header(this.text);
  final String text;
  static const Color primaryBlue = Color(0xFF0A63C9);
  @override
  Widget build(BuildContext context) => Expanded(
        child: Text(
          text,
          style: const TextStyle(color: primaryBlue, fontSize: 16, fontWeight:
FontWeight.w700),
        ),
      );
}

/// ---------- Table (PID/AT/BT/WT/TAT) ----------
class _ResultGrid extends StatelessWidget {
  const _ResultGrid({required this.rows, required this.perProc});
  final List<ResultRow> rows;
  final List<_PerProc> perProc;

  static const Color primaryBlue = Color(0xFF0A63C9);

  @override
  Widget build(BuildContext context) {
    final map = {for (final m in perProc) m.pid: m};

    final list = rows.isEmpty
        ? List<ResultRow>.filled(5, const ResultRow(pid: '', arrival: 0, burst: 0))
        : rows;

    return Column(
      children: List.generate(list.length, (i) {
        final r = list[i];
        final m = map[r.pid];
        final isLast = i == list.length - 1;
```

```
      return Column(
        children: [
          Container(
            height: 44,
            decoration: const BoxDecoration(
              border: Border(top: BorderSide(color: primaryBlue, width: 1.2)),
            ),
            child: Row(
              children: [
                Expanded(child: _cell(r.pid)),
                Expanded(child: _cell('${r.arrival}')),
                Expanded(child: _cell('${r.burst}')),
                Expanded(child: _cell(m == null ? '' : '${m.wt}')),
                Expanded(child: _cell(m == null ? '' : '${m.tat}')),
              ],
            ),
          ),
          if (isLast) const Divider(height: 1.2, thickness: 1.2, color: primaryBlue),
        ],
      );
    }),
  );
}

Widget _cell(String text) => Align(
      alignment: Alignment.centerLeft,
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 8),
        child: Text(
          text,
          overflow: TextOverflow.ellipsis,
          style: const TextStyle(
            color: Color(0xFF0A63C9),
            fontSize: 16,
            fontWeight: FontWeight.w600,
          ),
        ),
      ),
    );
}
```

```
class _PerProc {
  final String pid;
  final int wt;
  final int tat;
  const _PerProc({required this.pid, required this.wt, required this.tat});
}

/// ---------- Pretty Gantt Chart (single row) ----------
class _PrettyGanttChart extends StatelessWidget {
  const _PrettyGanttChart({
    required this.result,
    this.unitWidth = 32,
    this.barHeight = 30,
    this.showSwitchMarkers = true,
  });

  final List<Map<String, dynamic>> result; // [{pid,start,end}]
  final double unitWidth;
  final double barHeight;
  final bool showSwitchMarkers;

  @override
  Widget build(BuildContext context) {
    if (result.isEmpty) return const SizedBox.shrink();

    final data = [...result]..sort((a, b) => (a['start'] as int).compareTo(b['start'] as int));
    final start = data.first['start'] as int;
    final end = data.last['end'] as int;
    final total = (end - start).clamp(1, 1 << 30);

    // context switch times
    final switches = <int>[];
    for (int i = 1; i < data.length; i++) {
      if (data[i]['pid'] != data[i - 1]['pid']) {
        switches.add(data[i]['start'] as int);
      }
    }

    final chartWidth = unitWidth * total;

    return SingleChildScrollView(
      scrollDirection: Axis.horizontal,
```

```
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // GRID lines
          SizedBox(width: chartWidth, height: 8, child: _GridLineRow(start: start, end:
end, unitWidth: unitWidth)),
          const SizedBox(height: 2),

          // Gantt bars row (rounded gradient)
          SizedBox(
            width: chartWidth,
            height: barHeight,
            child: Stack(
              children: [
                for (final seg in data)
                  Positioned(
                    left: (seg['start'] - start) * unitWidth,
                    top: 0,
                    child: _CapsuleBar(
                      width: (seg['end'] - seg['start']) * unitWidth,
                      height: barHeight,
                      label: seg['pid'] as String,
                      color: _baseColor(seg['pid'] as String),
                    ),
                  ),

                // Context switch markers as red thin lines + dots
                if (showSwitchMarkers)
                  for (final t in switches)
                    Positioned(
                      left: (t - start) * unitWidth - 0.5,
                      top: 0,
                      child: Column(
                        children: [
                          Container(width: 1.5, height: barHeight, color: Colors.redAccent),
                          const SizedBox(height: 3),
                          Container(width: 6, height: 6, decoration: const
BoxDecoration(color: Colors.redAccent, shape: BoxShape.circle)),
                        ],
                      ),
                    ),
              ],
```

```
          ),
        ),

        const SizedBox(height: 8),

        // Time labels grey band
        _BottomBand(start: start, end: end, unitWidth: unitWidth),
      ],
    ),
  );
 }

 static Color _baseColor(String pid) {
  const palette = <Color>[
   Color(0xFF1E88E5), // blue
   Color(0xFF43A047), // green
   Color(0xFFFB8C00), // orange
   Color(0xFF8E24AA), // purple
   Color(0xFFE53935), // red
   Color(0xFF00897B), // teal
   Color(0xFF6D4C41), // brown
   Color(0xFF3949AB), // indigo
  ];
  return palette[pid.hashCode.abs() % palette.length];
 }
}

class _GridLineRow extends StatelessWidget {
 const _GridLineRow({required this.start, required this.end, required
this.unitWidth});
 final int start;
 final int end;
 final double unitWidth;

 @override
 Widget build(BuildContext context) {
  final ticks = end - start;
  return Row(
   children: List.generate(ticks, (i) {
    final isMajor = (i + start) % 5 == 0;
    return Container(
     width: unitWidth,
```

```
        decoration: BoxDecoration(
          border: Border(
            right: BorderSide(
              color: Colors.grey.withOpacity(isMajor ? 0.45 : 0.22),
              width: isMajor ? 1.2 : 0.8,
            ),
          ),
        ),
      );
    }),
  );
 }
}

class _CapsuleBar extends StatelessWidget {
 const _CapsuleBar({
  required this.width,
  required this.height,
  required this.label,
  required this.color,
 });

 final double width;
 final double height;
 final String label;
 final Color color;

 @override
 Widget build(BuildContext context) {
  final gradient = LinearGradient(
   colors: [
    HSLColor.fromColor(color).withLightness(0.65).toColor(),
    HSLColor.fromColor(color).withLightness(0.50).toColor(),
    HSLColor.fromColor(color).withLightness(0.40).toColor(),
   ],
   begin: Alignment.centerLeft,
   end: Alignment.centerRight,
  );

  return Container(
   width: width,
   height: height,
```

```
      decoration: BoxDecoration(
        gradient: gradient,
        borderRadius: BorderRadius.circular(height / 2),
        boxShadow: [
          BoxShadow(
            color: color.withOpacity(0.28),
            blurRadius: 8,
            offset: const Offset(0, 3),
          ),
        ],
      ),
      alignment: Alignment.center,
      padding: const EdgeInsets.symmetric(horizontal: 8),
      child: FittedBox(
        fit: BoxFit.scaleDown,
        child: Text(
          label,
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
          style: const TextStyle(color: Colors.white, fontWeight: FontWeight.w700),
        ),
      ),
    );
  }
}

class _BottomBand extends StatelessWidget {
  const _BottomBand({required this.start, required this.end, required
this.unitWidth});
  final int start;
  final int end;
  final double unitWidth;

  @override
  Widget build(BuildContext context) {
    final ticks = end - start + 1;
    return SizedBox(
      height: 26,
      child: Row(
        children: List.generate(ticks, (i) {
          final v = start + i;
          return Container(
```

```
          width: unitWidth,
          color: Colors.grey.shade700,
          alignment: Alignment.center,
          child: Text(
           '$v',
           style: const TextStyle(color: Colors.white, fontSize: 11, fontWeight:
FontWeight.w600),
          ),
         );
       }),
     ),
   );
  }
}
```

Module 7. gantt_chart.dart

```
import 'package:flutter/material.dart';

class GanttChart extends StatelessWidget {
  final List<Map<String, dynamic>> result;

  const GanttChart({super.key, required this.result});

  @override
  Widget build(BuildContext context) {
    final int startTime = result.first['start'];
    final int endTime = result.last['end'];
    final int totalTime = endTime - startTime;

    // Context switches detection
    List<Map<String, dynamic>> switches = [];
    for (int i = 1; i < result.length; i++) {
      if (result[i]['pid'] != result[i - 1]['pid']) {
        switches.add({
          'time': result[i]['start'],
          'from': result[i - 1]['pid'],
          'to': result[i]['pid'],
        });
      }
    }

    return Column(
```

```
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      // Time markers
      SingleChildScrollView(
        scrollDirection: Axis.horizontal,
        child: Row(
          children: List.generate(totalTime + 1, (i) {
            return Container(
              width: 20,
              alignment: Alignment.center,
              child: Text('${startTime + i}', style: const TextStyle(fontSize: 10)),
            );
          }),
        ),
      ),

      const SizedBox(height: 4),

      // Process execution row
      SingleChildScrollView(
        scrollDirection: Axis.horizontal,
        child: Row(
          children: result.map((item) {
            final duration = item['end'] - item['start'];
            return Container(
              width: duration * 20.0,
              height: 40,
              alignment: Alignment.center,
              decoration: BoxDecoration(
                color: Colors.primaries[item['pid'].hashCode % Colors.primaries.length],
                border: Border.all(color: Colors.black, width: 1.2),
              ),
              child: Text(item['pid'],
                  style: const TextStyle(color: Colors.white, fontWeight:
FontWeight.w600)),
            );
          }).toList(),
        ),
      ),

      const SizedBox(height: 8),
```

```
        // Context switch row
        SingleChildScrollView(
          scrollDirection: Axis.horizontal,
          child: Row(
            children: List.generate(totalTime, (i) {
              final currentTime = startTime + i;
              final switchHere = switches.any((s) => s['time'] == currentTime);
              return Container(
                width: 20,
                height: 20,
                alignment: Alignment.center,
                decoration: BoxDecoration(
                  color: switchHere ? Colors.redAccent : Colors.transparent,
                  border: Border.all(color: Colors.grey.shade300),
                ),
                child: switchHere
                    ? const Icon(Icons.swap_horiz, size: 12, color: Colors.white)
                    : null,
              );
            }),
          ),
        ),
        const SizedBox(height: 4),
        const Text('🔄 Context switches shown in red',
            style: TextStyle(fontSize: 12, fontWeight: FontWeight.w600)),
      ],
    );
  }
}
```

6.2. Snap Shots Of The Project:-

Image – Splash Screen (Welcome to SchedSim)

- The splash screen features a clean blue background symbolizing technology and clarity.
- A central microchip icon represents the core concept of CPU scheduling.
- The text "Welcome to" appears in a smooth, handwritten font for a friendly look.
- The bold title "SCHEDSIM" clearly displays the app's name and identity.
- Below, a short tagline describes the app's purpose — exploring CPU scheduling algorithms like FCFS, SJF, RR, and PS.
- A "Get Started" button at the bottom invites users to proceed into the app.
- The overall layout is simple, elegant, and user-friendly, ensuring smooth visual appeal.
- The blue and white color scheme creates a modern **and professional look.**

Fig. No. 6

Image – Algorithm Selection Page

- The screen title "Select Algorithm" appears at the top with a clean blue header for clarity and consistency.
- The page displays four neatly designed option cards — FCFS, SJF, RR, and PS — representing different CPU scheduling algorithms.
- Each option card is outlined with a blue border and rounded corners, giving a modern and minimalistic look.
- The algorithm names are written in full form followed by their short forms in parentheses for better understanding.
- A radio-button style selector on the right indicates the user's current selection.
- The overall layout uses a light background with blue accents, ensuring readability and visual balance.
- The design provides a simple and intuitive navigation experience, allowing users to quickly choose their desired algorithm.
- The interface follows a consistent theme with the splash screen, maintaining a cohesive visual identity throughout the app.

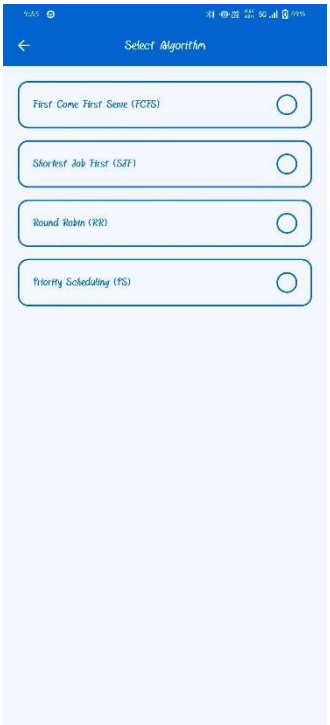

Fig. No. 7

Fig. No. 8

Image – Algorithm Selection Page

- The screen title "Select Algorithm" appears at the top with a clean blue header for clarity and consistency.
- The page displays four neatly designed option cards — FCFS, SJF, RR, and PS — representing different CPU scheduling algorithms.
- Each option card is outlined with a blue border and rounded corners, giving a modern and minimalistic look.
- The algorithm names are written in full form followed by their short forms in parentheses for better understanding.
- A radio-button style selector on the right indicates the user's current selection.
- The overall layout uses a light background with blue accents, ensuring readability and visual balance.
- The design provides a simple and intuitive navigation experience, allowing users to quickly choose their desired algorithm.
- The interface follows a consistent theme with the splash screen, maintaining a cohesive visual identity throughout the app.

Image – Process Input Form Page (FCFS Simulation Page)

- The screen displays the selected algorithm title "FCFS" clearly at the top of the page.
- A short instruction text — "Enter the process details below to start the simulation" — guides the user.
- Input fields are provided for each process with labels: PID, Arrival Time, and Burst Time.
- The layout is organized in a table-like format, making it easy to compare and enter values for multiple processes.
- The Submit button at the bottom allows users to begin the simulation after entering data.
- The interface maintains a consistent blue and white theme, ensuring clarity and readability.
- The design is simple, clean, and user-friendly, helping users focus on accurate process input before running the algorithm.

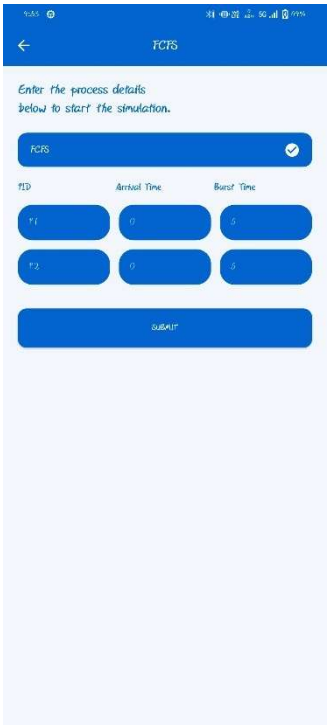

Fig. No. 9

Fig. No. 10

Image- Results Page (Output Screen – FCFS)

- The screen is titled **"RESULTS"**, clearly indicating the output section of the simulation.
- A short instruction line — *"Review the Gantt chart and detailed process metrics below"* — guides the user.
- The selected algorithm (e.g., **FCFS**) is shown inside a highlighted blue box for confirmation.
- A **tabular layout** displays each process along with its **Arrival Time**, **Burst Time**, **Waiting Time**, and **Turnaround Time**.
- Below the table, **average waiting time** and **average turnaround time** are automatically calculated and shown.
- A colorful **Gantt Chart** is displayed at the bottom, visually representing process execution order and timing.
- The use of different colors for each process in the chart helps in clear visualization and easy understanding.
- The design maintains a **consistent blue theme** with a clean, analytical, and user-friendly interface.

Image-Algorithm Information Page (Shortest Job First – SJF)

- The screen title clearly shows the selected algorithm name — **"Shortest Job First (SJF)"**.
- A short description below the title explains the logic — *"Executes the job with the smallest CPU burst next."*
- The line *"You selected: Shortest Job First (SJF)"* confirms the user's chosen scheduling algorithm.
- The next section asks the user to **enter the total number of processes**, preparing for input simulation.
- A **numeric text field** allows the user to input process count (e.g., 4).
- The **Submit button** at the bottom continues to the process input screen for data entry.
- The interface maintains a **consistent blue and white theme**, giving a professional and educational look.
- The page design is **minimal, user-friendly, and clearly structured**, helping users understand the purpose instantly.

Fig. No. 11

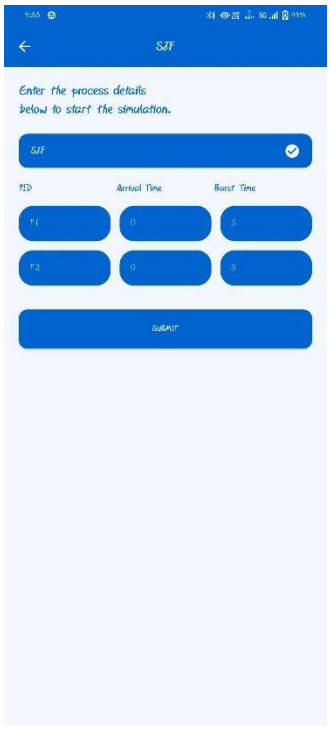

Fig. No. 12

Image-Process Input Page (Shortest Job First – SJF)

1. The page title clearly shows the selected algorithm as "SJF", indicating that the Shortest Job First method is being used.
2. An instruction line — "Enter the process details below to start the simulation" — guides the user to input process data.
3. The selected algorithm name (SJF) is displayed in a blue box, confirming the current mode.
4. The form displays labeled fields for Process ID (PID), Arrival Time, and Burst Time.
5. Each process row allows users to input respective values for simulation.
6. A Submit button at the bottom allows users to proceed after entering process details.
7. The screen uses a simple blue and white design, ensuring visual consistency with other app screens.
8. The UI is clean, organized, and user-friendly, helping users quickly enter process information without confusion.

Image-Results Page (Output Screen – SJF Algorithm)

- The screen is titled "RESULTS", indicating the output and analysis section of the simulation.
- It starts with an instruction line — "Review the Gantt chart and detailed process metrics below" — guiding users on what to observe.
- The selected algorithm, Shortest Job First (SJF), is displayed inside a highlighted blue box for clarity.
- The output table lists PID, Arrival Time, Burst Time, Waiting Time, and Turnaround Time for each process.
- Below the table, the app calculates and displays the Average Waiting Time and Average Turnaround Time automatically.
- The Gantt Chart visually represents the execution order of processes (e.g., P1, P2) based on shortest burst time.
- Each process in the Gantt chart is color-coded for clear differentiation and better visualization.
- The clean, blue-and-white themed interface ensures an organized, interactive, and educational output display.

Fig. No. 13

Fig. No. 14

Image-Algorithm Information Page (Round Robin – RR)

1. The page title clearly shows the selected algorithm — "Round Robin (RR)".
2. A short description under the title explains the logic — "Each process gets a fixed time quantum in round-robin fashion."
3. The confirmation text "You selected: Round Robin (RR)" assures the user of their chosen algorithm.
4. The section below prompts the user to enter the total number of processes to start the simulation.
5. A simple numeric input box is provided for entering process count (e.g., 4).
6. A Submit button is displayed below to proceed to the process input page.
7. The layout follows a consistent blue and white theme, maintaining harmony with other app screens.
8. The interface is clean, easy to use, and focused on clarity, helping users understand the next simulation step smoothly.

Image-Process Input Page (Shortest Job First – SJF)

1. The page title clearly shows the selected algorithm as "SJF", indicating that the Shortest Job First method is being used.
2. An instruction line — "Enter the process details below to start the simulation" — guides the user to input process data.
3. The selected algorithm name (SJF) is displayed in a blue box, confirming the current mode.
4. The form displays labeled fields for Process ID (PID), Arrival Time, and Burst Time.
5. Each process row allows users to input respective values for simulation.
6. A Submit button at the bottom allows users to proceed after entering process details.
7. The screen uses a simple blue and white design, ensuring visual consistency with other app screens.
8. The UI is clean, organized, and user-friendly, helping users quickly enter process information without confusion.

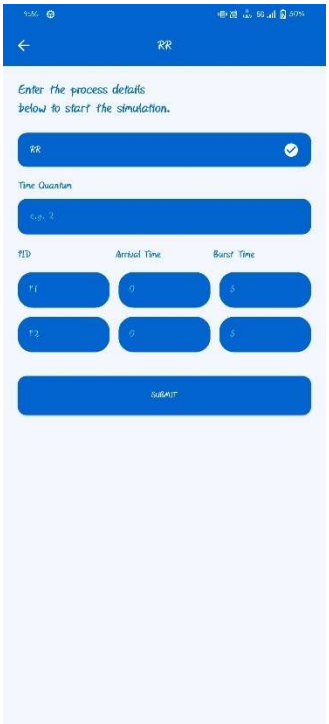

Fig. No. 15

Fig. No. 16

Image-Results Page (Output Screen – Round Robin Algorithm)

1. The screen heading "RESULTS" clearly represents the output and analysis phase of the Round Robin simulation.
2. A short instruction line guides the user — "Review the Gantt chart and detailed process metrics below."
3. The selected algorithm, Round Robin (RR), is highlighted in a blue box for clarity.
4. The output table displays each process with its Arrival Time, Burst Time, Waiting Time, and Turnaround Time.
5. The system calculates and shows the Average Waiting Time and Average Turnaround Time automatically below the table.
6. A colorful Gantt Chart visually represents the time slices of each process according to the Round Robin scheduling.
7. Each process in the chart is color-coded, allowing easy identification of CPU time allocation.
8. The blue-white color scheme and simple layout give the page a professional, clean, and educational appearance.

Image-Algorithm Information Page (Priority Scheduling – PS)

1. The page heading clearly mentions the selected algorithm — "Priority Scheduling (PS)".
2. A short description explains the concept — "Higher priority process runs first (preemptive/non-preemptive)."
3. A confirmation line — "You selected: Priority Scheduling (PS)" — reassures the user of the chosen scheduling type.
4. The section below prompts the user to enter the total number of processes to be simulated.
5. An input field allows the user to specify the process count (e.g., 4).
6. A Submit button is provided at the bottom for proceeding to the process detail entry screen.
7. The interface maintains the app's consistent blue and white theme, ensuring uniformity across all pages.
8. The layout is minimal, clear, and user-friendly, focusing on easy data entry and smooth navigation.

Fig. No. 17

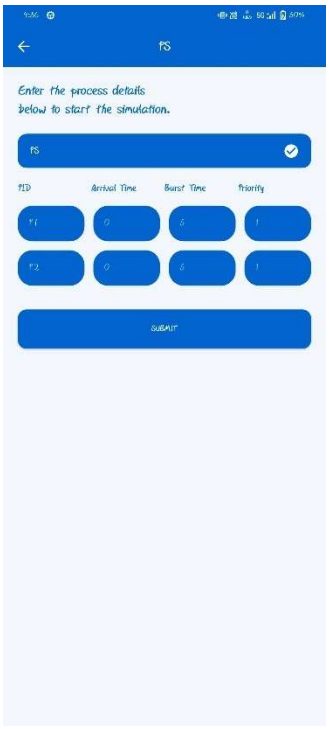

Fig. No. 18

Image-Process Input Page (Priority Scheduling – PS)

1. The screen title clearly shows the selected algorithm as "Priority Scheduling (PS)", ensuring the user knows which scheduling technique is active.
2. The top instruction — "Enter the process details below to start the simulation." — guides users for accurate data entry.
3. The selected algorithm name (PS) is displayed in a blue box to confirm the current mode.
4. The form provides labeled fields for each process:
    - PID (Process ID)
    - Arrival Time
    - Burst Time
    - Priority (higher value indicates higher importance)
5. The input layout allows users to add multiple processes and their respective details.
6. A large Submit button is placed below to proceed with the simulation after entering all values.
7. The page follows a clean blue-white theme for readability and uniform visual design.
8. The interface is designed for clarity, simplicity, and quick data input, suitable for both students and beginners learning scheduling algorithms.

Image-Results Page (Priority Scheduling – PS)

1. The page heading "RESULTS" clearly indicates the final output screen of the simulation.
2. A short line instructs users — "Review the Gantt chart and detailed process metrics below."
3. The selected algorithm, Priority Scheduling (PS), is highlighted in a blue selection box for clarity.
4. The result table displays detailed process metrics including —
    - Process ID (PID)
    - Arrival Time
    - Burst Time
    - Waiting Time
    - Turnaround Time
5. The system automatically calculates and displays the Average Waiting Time and Average Turnaround Time below the table.
6. A visually rich Gantt Chart represents the execution order of processes based on their priorities.
7. Each process block in the Gantt chart is color-coded for clear differentiation.
8. A Download PDF button allows users to export and save the simulation results conveniently.
9. The overall layout is clean, minimal, and educational, providing an excellent visual understanding of process scheduling outcomes.
10. The consistent blue and white theme maintains design harmony across all app screens.

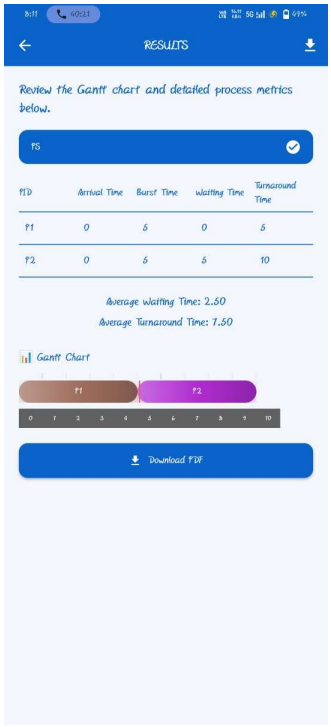

Fig. No. 19

7. CONCLUSION AND FUTURE SCOPE

The **CPU Scheduler Simulator App** was successfully developed to demonstrate the working of various **CPU scheduling algorithms** in a simple and interactive way. The application allows users to enter process details, simulate execution, and visualize results through **tables and Gantt charts**.

It was entirely designed and implemented using **Flutter**, and all UI layouts were pre-planned using **Canva** to ensure a clean and modern interface.

This project helped in understanding the **practical concepts of operating systems**, especially CPU scheduling techniques such as **FCFS**, **SJF**, **Priority**, and **Round Robin**. The app effectively calculates and displays important metrics like **waiting time** and **turnaround time**, helping users analyze algorithm performance visually.

In the future, this project can be further extended by adding features such as:

- Dynamic process addition during execution,
- Graphical analysis of CPU utilization, and
- Web-based version for broader accessibility.

Overall, the project achieved its goal of creating an educational and user-friendly tool for visualizing CPU scheduling algorithms.

8. REFERENCES / BIBLIOGRAPHY

- **Flutter Documentation** – https://flutter.dev/docs → Used for understanding Flutter widgets, layouts, and UI implementation.
- **Dart Programming Language Guide** – https://dart.dev/guides → Referred for learning syntax, state management, and asynchronous programming concepts.
- **Canva Design Platform** – https://www.canva.com → Used to design the complete UI layout and visual elements of the app.

- **GeeksforGeeks – CPU Scheduling Algorithms** – https://www.geeksforgeeks.org → Referred to understand theoretical concepts and algorithm logic for FCFS, SJF, Priority, and Round Robin.
- **Operating System Concepts (by Silberschatz, Galvin, and Gagne)** → Used as a theoretical reference for CPU scheduling fundamentals.
- **YouTube Tutorials** → Referred for practical demonstrations of Flutter UI development and scheduling algorithm visualizations.
- **Stack Overflow** – https://stackoverflow.com → Helped in solving implementation and debugging issues during development.

9. AI-Generated Project Elaboration/Breakdown Report

This project, titled **SchedSim: An Intelligent CPU Scheduling Algorithm App**, represents a comprehensive effort to design, develop, and implement a mobile-based simulator for CPU scheduling algorithms. The report outlines the problem statement, scope, design, development, requirements, and implementation phases in detail. Below is the AI-generated breakdown of the project:

---

1. Introduction & Problem Statement

The project addresses the challenge of understanding CPU scheduling algorithms such as **FCFS, SJF, Round Robin, Priority Scheduling, SRTF, and Multilevel Queue**. Traditional learning methods are text-heavy and lack visualization, making it difficult for students to grasp execution flow.

SchedSim bridges this gap by offering a **Flutter-based mobile application** that provides:

- Real-time process input and scheduling simulation
- Automated computation of **Waiting Time (WT)** and **Turnaround Time (TAT)**
- Dynamic **vertical Gantt chart visualization** optimized for mobile screens

---

2. Scope of Work

The system introduces innovations beyond existing simulators:

- **Interactive UI** with algorithm selection, info pages, input forms, and results visualization
- **Dynamic Gantt charts** with distinct colors, context-switch markers, and animations
- **Educational focus**, making complex scheduling concepts easy to understand
- **Extensible architecture**, allowing future integration of preemptive algorithms and export features

---

3. Project Analysis

Existing simulators are console-based or web-only, focusing on numerical results without visualization. This project fills the gap by:

- Providing **mobile accessibility** and offline functionality
- Offering **step-by-step guided simulation**
- Ensuring **educational clarity** with both tabular and graphical outputs

- Supporting **real-time experimentation** for students and educators

---

4. Project Definition & Overview

The app is defined as a **learning and demonstrative tool** for Operating Systems. It enables:

- Easy process input (PID, Arrival Time, Burst Time, Priority, Quantum)
- Automatic computation of WT and TAT
- Dynamic visualization of execution order
- Comparison of algorithms in real-time

The project overview emphasizes its role as an **interactive educational platform**, transforming abstract theory into practical experience.

---

5. System Design

- **User Navigation System:** Smooth flow from algorithm selection → info → input → results
- **Algorithm Management Module:** Handles structured process data and executes scheduling logic
- **Dynamic User Interface:** Flutter-based, responsive, interactive, and visually appealing
- **Planning & Requirements Analysis:** Clear goals, validation mechanisms, and modular design

6. Development

- **Front-End:** Flutter/Dart UI with responsive layouts, animations, and navigation
- **Back-End:** Local computation of scheduling algorithms without external servers
- **Tools & Technologies:** Dart, Flutter, VS Code, GitHub, Postman, Android Studio
- **Testing & QA:** Unit testing of widgets, algorithm verification, integration testing, bug fixing

---

7. Requirements

- **Functional:** Algorithm selection, process input, computation, Gantt chart visualization, error handling
- **Non-Functional:** Performance, scalability, usability, reliability, compatibility, security (future scope)
- **General Description:** Offline-first architecture with potential cloud integration, deployment via APK/Play Store, comprehensive documentation

---

8. Design Documentation

- **Revision Tracking on GitHub:** Version control, collaboration, branching, commit history, cloud backup

- **UI Design:** Canva-based planning, consistent color scheme, responsive layouts, visual consistency
- **ER Diagram & DFDs:** Structured representation of database and process flows

---

9. Implementation

- **Code Implementation:** Dart-based scheduling logic, modularized structure, error handling
- **Snapshots:** Screens of algorithm selection, input forms, results pages, and Gantt charts for FCFS, SJF, RR, and Priority Scheduling

---

10. Conclusion & Future Scope

The project successfully delivers an **interactive CPU scheduling simulator** that enhances conceptual learning. Future scope includes:

- Adding **preemptive algorithms** (SRTF, Priority Preemptive)
- Exporting results (PDF/PNG)
- Integrating **cloud-based storage and analytics**
- Developing **interactive learning modules** for deeper engagement